

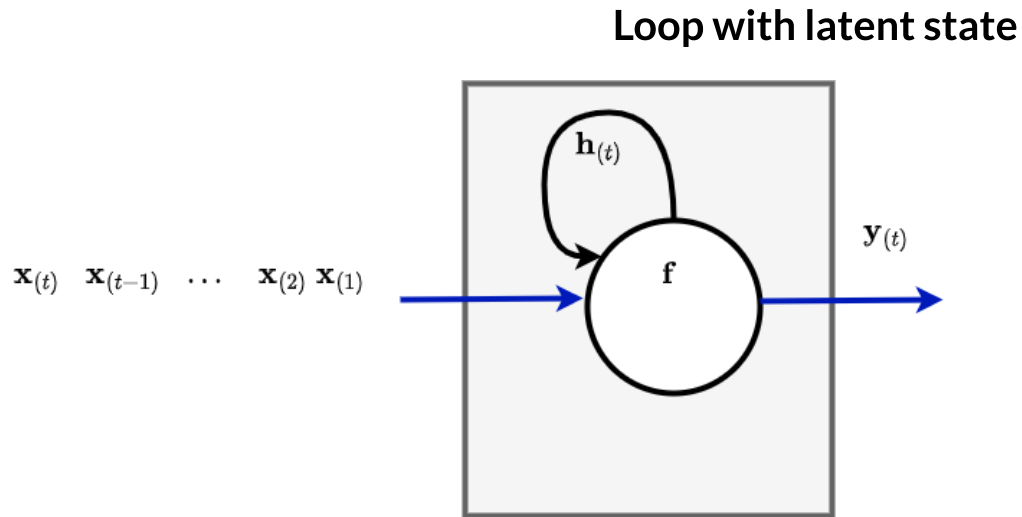
Dealing with Sequences: Recurrent Neural Network (RNN) layer

For a function that takes sequence $\mathbf{x}^{(i)}$ as input and creates sequence $\mathbf{y}$ as output we had two choices for implementing the function.

The RNN implements the function as a "loop"

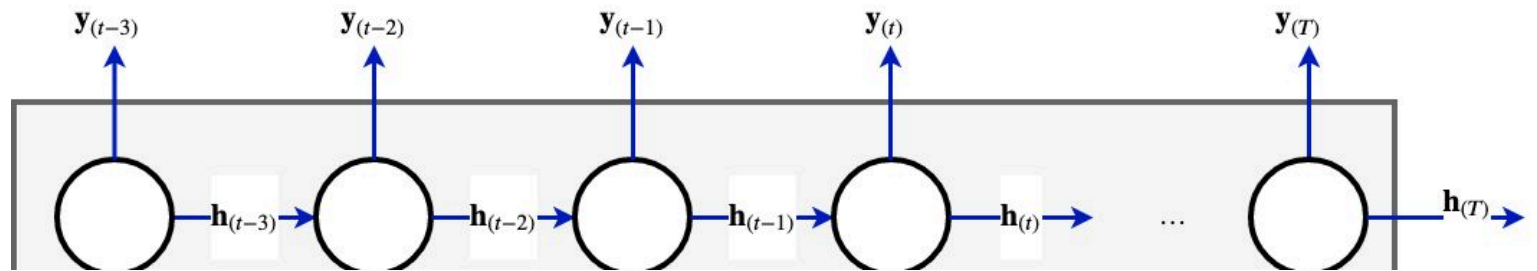
- A function that taking a **single** $\mathbf{x}_{(t)}$ as input a time
- Outputting $\mathbf{y}_{(t)}$
- Using a "latent state" $\mathbf{h}_{(t)}$ to summarize the prefix $\mathbf{x}_{(1 \dots t)}$
- Repeat in a loop over t

$p(\mathbf{h}_{(t)}|\mathbf{x}_{(t)}, \mathbf{h}_{(t-1)})$ latent variable $\mathbf{h}_{(t)}$ encodes $[\mathbf{x}_{(1)} \dots \mathbf{x}_{(t)}]$
 $p(\mathbf{y}_{(t)}|\mathbf{h}_{(t)})$ prediction contingent on latent variable



"Unrolling" the loop makes it equivalent to a multi-layer network

RNN unrolled



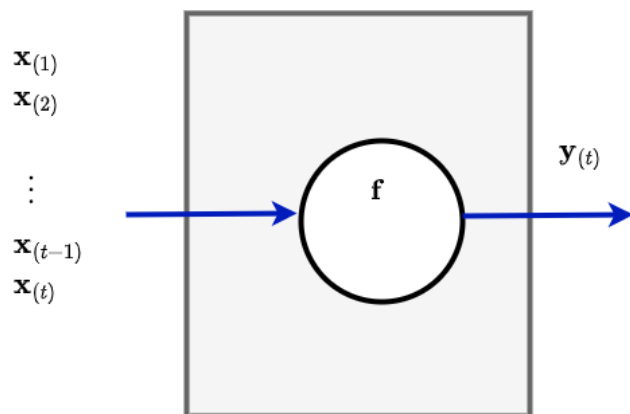
Transformer variants

Encoder style

The alternative to the loop was to create a "direct function"

- Taking a **sequence** $\mathbf{x}_{(1..t)}$ as input
- Outputting $\mathbf{y}_{(t)}$

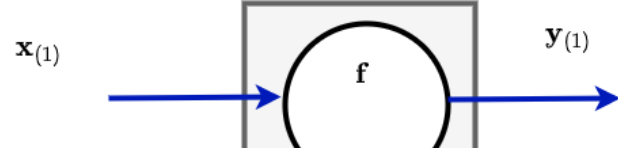
Direct function



In order to output the sequence $\mathbf{y}_{(1)} \dots \mathbf{y}_{(T)}$ we create T copies of the function (one for each $\mathbf{y}_{(t)}$)

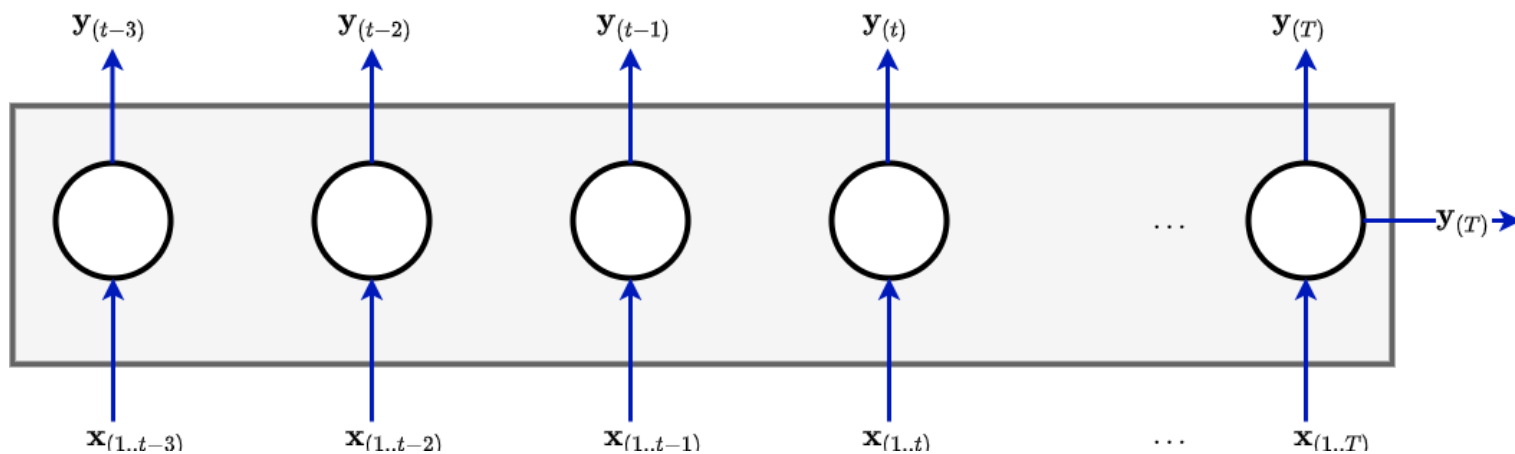
- computes each $\mathbf{y}_{(t)}$ in **parallel**, not sequentially as in the loop

Direct function, in parallel (masked input)



The parallel units constitute a *Transformer Encoder*

Transformer Encoder (causal masked input)



Compared to the unrolled RNN, the Transformer Encoder

- Takes a **sequence** $\mathbf{x}_{(1..t)}$ as input
 - Because $\mathbf{y}_{(t)}$ is computed as a *direct* function of the prefix $\mathbf{x}_{(1..t)}$ rather than recursively
- Has **no** latent state: output is a direct function of the input sequence
- Has **no** data (e.g., $\mathbf{h}_{(t)}$) passing from the computation between time steps (e.g., from t to $(t + 1)$)
- Outputs generated in parallel, not sequentially
- No gradients flowing backward over time

With this architecture, we can compute more general functions than the RNN

- where each $\mathbf{y}_{(t)}$ depends on the entire $\mathbf{x}_{(1..T)}$ rather than a prefix $\mathbf{x}_{(1..t)}$

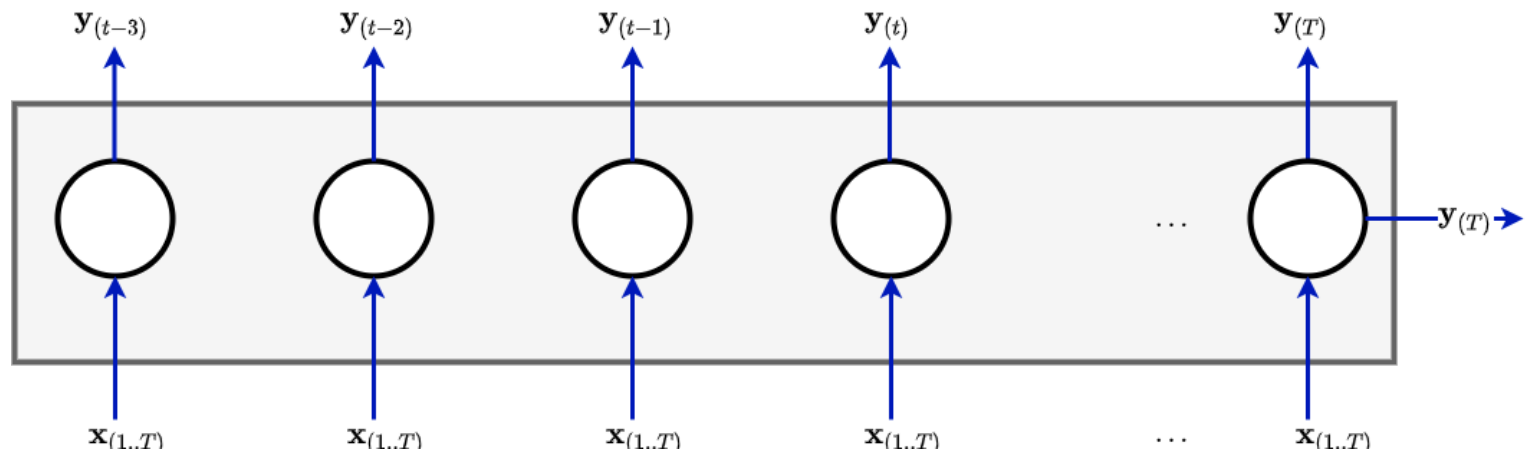
Direct function, in parallel (un-masked input)

$\mathbf{x}_{(1 \dots T)}$



$\mathbf{y}_{(1)}$

Transformer Encoder (unmasked input)



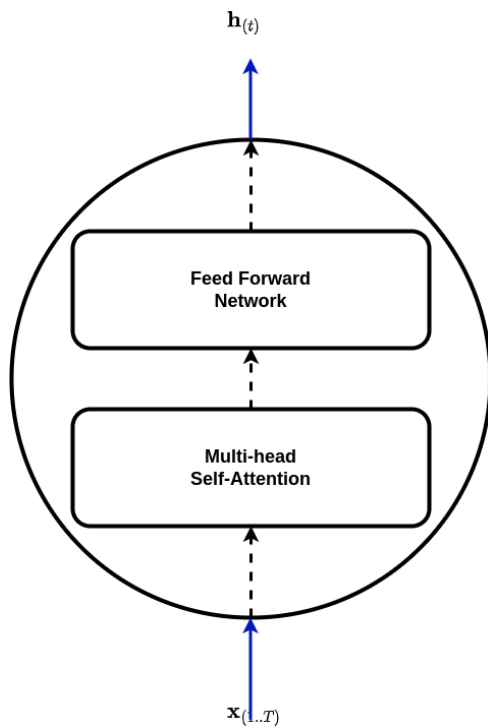
Note that we use *unmasked* Self Attention for the Encoder

- it is OK for the encoder to access all input positions $\mathbf{x}_{(1:\bar{T})}$

This is useful, for example, when the meaning of a word depends on its *entire* context.

Here is a diagram of an Encoder style Transformer block.

Transformer layer: Encoder style



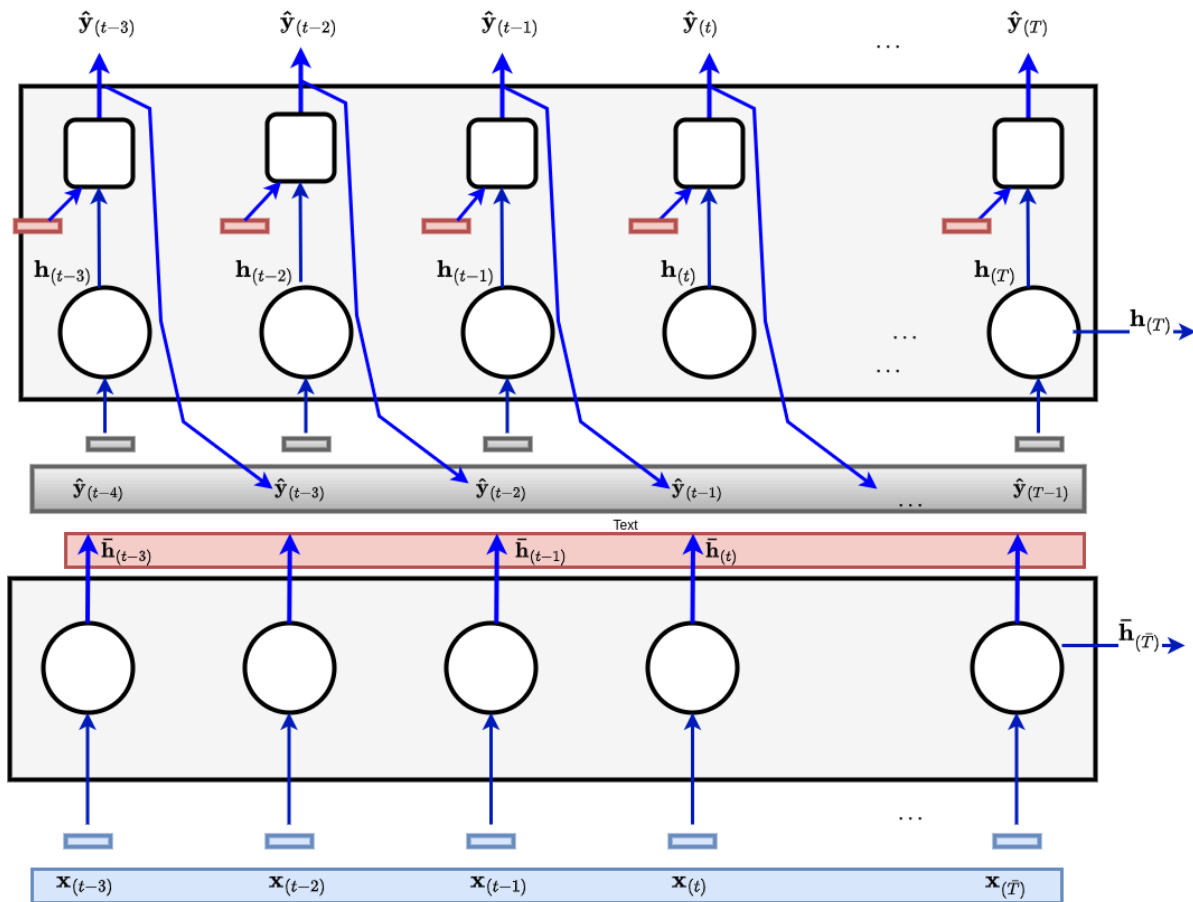
Decoder style within Encoder/Decoder

It is common to use two Transformers in an Encoder/Decoder configuration.

Refer back to our [Attention module \(Intro to Attention.ipynb#Attention\)](#)

- used to motivate Attention
- through several steps
 - we modified a pair of RNN's (Encoder and Decoder)
 - into a pair of direct function modules
 - which form the basis for the Transformer

Encoder/Decoder with Cross Attention and Self Attention (Encoder/Decoder)



The Encoder part of the pair

- has full visibility to the input sequence $\mathbf{x}_{(1 \dots \bar{T})}$
 - via un-masked Self-Attention
- it transforms the input into sequence $\bar{\mathbf{h}}_{(1 \dots \bar{T})}$
 - which is made available to the Decoder when generated each element of the Output $\hat{\mathbf{y}}_{(t)}$

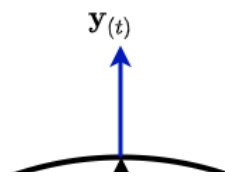
The Decoder part of the pair

- can attend to the Encoder Input $\bar{\mathbf{h}}_{(1..\bar{T})}$ when generating any Output $\hat{\mathbf{y}}_{(t)}$
- the Output is produced auto-regressively
 - so Output at position t is a function of
 - $\hat{\mathbf{y}}_{(1..t-1)}$: the Output generated thus far
 - the encoded input $\bar{\mathbf{h}}_{(1..\bar{T})}$

The Decoder uses two types of Attention

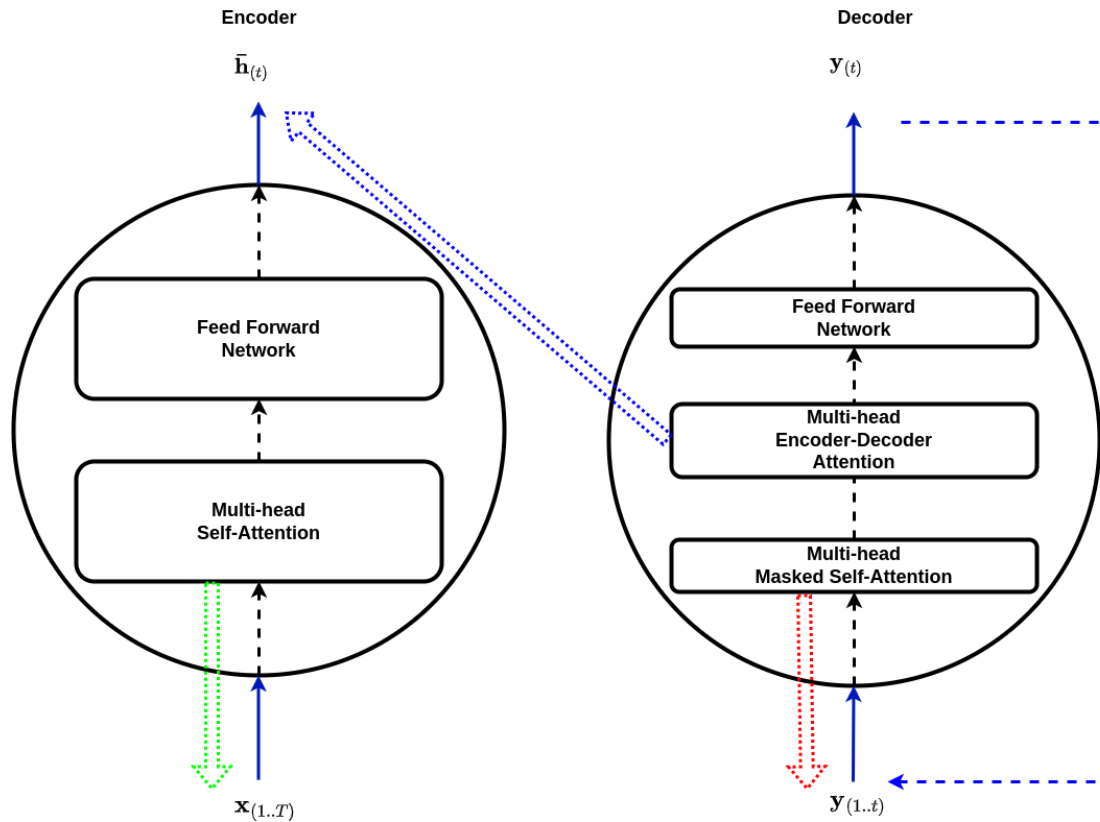
- Attention to the Encoder output $\bar{\mathbf{h}}_{(1..\bar{T})}$: Encoder/Decoder Cross Attention
- Self-Attention to the Output generated thus far $\hat{\mathbf{y}}_{(1..t-1)}$
 - Masked Self-Attention
 - Self-Attention: attends to its own input
 - Masked: access only to prefix $\hat{\mathbf{y}}_{(1..t-1)}$ rather than full $\hat{\mathbf{y}}_{(1..T)}$

Transformer Layer (Decoder)



The combined Encoder-Decoder Transformer diagram looks like this

Transformer Layer (Encoder/Decoder)



Explanation of diagram

- The Encoder uses Self-attention (**wide Green arrow**) to attend to input sequence x
- The Decoder uses Masked Self-attention (**wide Red arrow**) to attend to its input
 - It's input is the prefix of the output sequence y
 - Limited to prefix of length t by masking
- The Decoder uses Cross Attention (between Encoder and Decoder) (**wide Blue arrow**)
 - To enable Decoder to focus on which Encoder latent state $\bar{h}_{(t)}$ to attend to
- The dotted (**thin Blue arrow**) indicates that the output $\hat{y}_{(t)}$ is appended to the input that is available when generating $\hat{y}_{(t+1)}$

Note that the Decoder is recurrent (auto-regressive; generative)

- **it generates a single output at a time**
- **unlike the Encoder, which generates all outputs (i.e., "encodings") in parallel**

During training, at step t

- the entire Target $\mathbf{y}_{(1..T)}$ is available as input
- but Causal masking ensures that only $\mathbf{y}_{(1..t-1)}$ is visible
- so the *available* input at step t is $\mathbf{y}_{(1..t-1)}$
- note that Training time input is $\mathbf{y}_{(1..t-1)}$ not $\hat{\mathbf{y}}_{(1..t-1)}$
 - Teacher forcing to prevent cascading errors
 - stops errors at step $t - 1$ from affecting predictions at subsequent steps

Decoder Style (stand-alone)

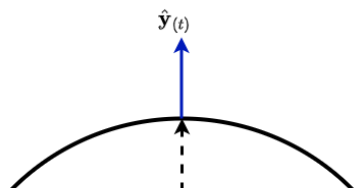
The Decoder can also be used independently from an Encoder.

- eliminate the Cross-Attention block

This style decoder is similar to an Encoder

- but uses masked Self-Attention rather than unrestricted Attention

Transformer layer: Decoder style

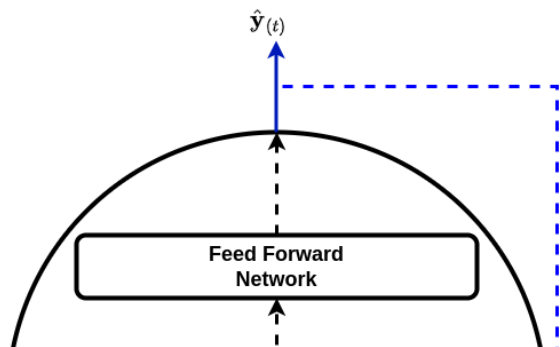


A Decoder usually operates in an *auto regressive* manner

- it has no initial input
 - Technically: it has a special "start" token
- the output $\hat{y}_{(t)}$ of time step t is appended to the input
 - so the input at time step t is

$$\hat{y}_{(1..t-1)}$$

Transformer layer: Decoder style



When this occurs, the Encoder at time step t can only attend to a *prefix* of $\hat{\mathbf{y}}_{(1..T)}$

$$\hat{\mathbf{y}}_{(1..t-1)}$$

- it can't refer to an input that will only be available (at *inference time*) in the future !
- Recall
 - During training, at step t
 - the entire Target $\mathbf{y}_{(1..T)}$ is available as input
 - masking prevents peeking into the future

This is implemented by a form of Attention called Masked Self-Attention

- $\hat{\mathbf{y}}_{(1..T)}$ is masked to make visible only the prefix $\hat{\mathbf{y}}_{(1..t-1)}$

Use cases for each variant of Transformer

The Transformer for the Encoder and Decoder of an Encoder/Decoder Transformer are slightly different.

They can also be used individually as well as in pairs.

It's important to understand the differences in order to know when to use each individually.

Encoder/Decoder uses

The Encoder/Decoder acts as a function

- from Input, processed by the Encoder
- to Target, processed by the Decoder

This is a natural architecture for Sequence to Sequence tasks.

Encoder only uses

The Encoder side of the pair does not restrict the order in which it's inputs are accessed.

- Self-attention without causal masking

Thus the Encoder output at *each* position is a function of the input at *all* positions.

This is valuable for tasks that require a context-sensitive representation of each input element.

For example: the meaning of the word "it" changes with a small change to a subsequent word in the following sentences:

- **"The animal didn't cross the road because it was too tired"**
- **"The animal didn't cross the road because it was too wide"**

Some tasks with this characteristic are

- Sentiment
- Masked Language Modeling: fill-in the masked word
- Semantic Search
 - compare a summary of the sequence that is the context-sensitive representation of
 - query sentence
 - document sentences
 - Each summary is a kind of sentence embedding
 - Summary
 - pooling over each word
 - final token

It is often the case that special tokens are added to the input of an Encoder style transformer.

- Designating a special role for this token, compared to the other tokens in the sequence
- For example <CLS> ("Classification") is the single token used as input to a subsequent Classifier layer

Thus Encoder style Transformers are usually used as the first "layer" of a multi-layer network

- later layers being, e.g., task-specific "heads"

Decoder only uses

A Decoder style Transformer

- looks like the Decoder side of the Encoder-Decoder
- *without* Cross-Attention, since there is no Encoder

One notable aspect of the Decoder is its auto-regressive behavior

- Initial input is empty
- Output $\hat{y}_{(t-1)}$ is appended to the Decoder inputs available at step t .
- step t input: $\hat{y}_{([0:t-1])}$

Thus, a Decoder only Transformer is useful for completely *generative* task

- create sequence output
- from no input

One can modify a Decoder only Transformer to implement a function from Input to Target

- just like an Encoder/Decoder
- by initializing the Decoder input to the function input sequence $\mathbf{x}_{(0..\bar{T})}$

Thus, a Decoder only Transformer become similar in function to an Encoder/Decoder Transformer.

- but with half the parameters (since no Encoder)

Stacked Transformer

Just as with many other layer types (e.g., RNN), we may stack Transformer layers.

- Each layer creating alternate representations of the input of increasing complexity

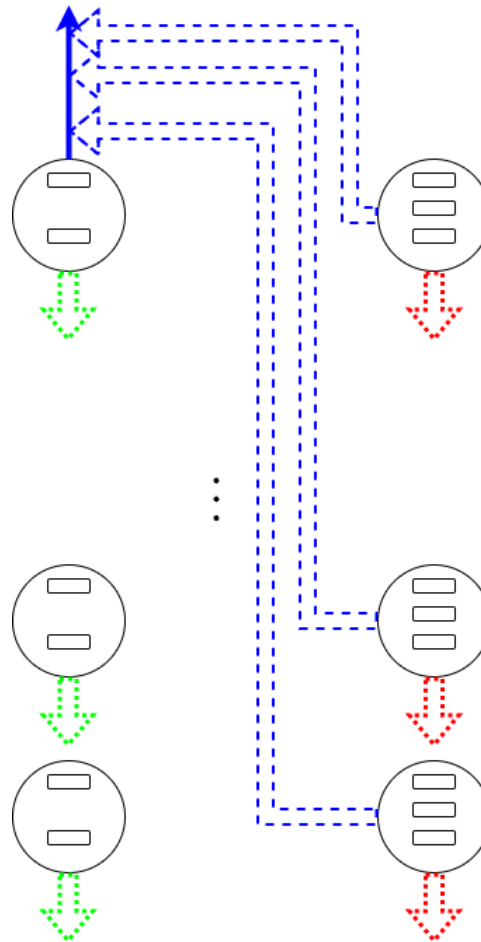
In fact, stacking $N > 1$ Transformer layers is typical.

$N = 6$ was the choice of the original paper.

Note that this is still an Encoder/Decoder

- so the *final* output of the Encoder is attended to by *each* layer of the Decoder.

Stacked Transformer Layers (Encoder/Decoder)



Advantages of a Transformer compared to an RNN (w/o Attention)

We will illustrate some of the advantages that Attention brings to the Transformer

- high degree of parallelism possible
- facilitates longer sequences
 - Vanishing Gradient no longer an issue
- facilitates more complex computation (via deeper stack of layers)

The price for this

- potentially higher number of operations

We compare a Decoder style (Auto-regressive) Transformer to an RNN without Attention.

Number of sequential steps

Let's consider the output sequence $\hat{\mathbf{y}}_{(1:T)}$ produced by an Auto-regressive Decoder.

During training, we use Teacher Forcing

- so that $\hat{\mathbf{y}}_{(t)}$ depends on true target $\mathbf{y}_{(1:t-1)}$
- rather than $\hat{\mathbf{y}}_{(1:t-1)}$

Note

Even though $\hat{\mathbf{y}}_{(t)}$ is not fed into the input of the next step

- it still must be computed
- as the Loss for step t depends on it

The RNN (without Attention) must produce each output $\hat{y}_{(t)}$ sequentially

- even though the full target sequence $y_{(1:T)}$ is available in the training examples
- because $\hat{y}_{(t)}$ depends on the previous latent state $\mathbf{h}_{(t-1)}$

Thus an RNN takes T sequential steps to process an entire sequence of length T .

- spanning T training examples

But, because of Attention, a Transformer

- can produce the output $\hat{y}_{(t)}$ of all T training examples $1 \leq t \leq T$
- *simultaneously*
- given the available of parallel compute

That is, to produce T outputs

- the Transformer takes 1 step
- the RNN takes T steps

This means that

- training time using a Transformer
- can be reduced compared to an RNN
- via parallel processing
 - multiple GPU's
 - because we can compute the loss of all T examples in a sequence
 - in one parallel time step

We can leverage this time advantage by

- **Increasing the number of Transformer layers in the stack**
- **So each time step now takes longer**
 - **the blocks in the stack are processed sequentially**
 - **but this is a constant (finite number of layers)**
- **Trading off increased depth for a more powerful (deeper) stack**

Path length

The *Path length* is the distance that the Loss Gradient needs to travel backwards during Back Propagation.

At each step, the gradient is subject to being diminished or increased (Vanishing/Exploding gradients).

Since the Transformer operates in parallel across positions, this is $\mathcal{O}(1)$.

It is $\mathcal{O}(T)$ for the RNN due to the sequential computation.

The constant path length is critical to the success of the Transformer

- The query used for the input at position t can access all prior positions $t' \leq t$ at the same cost
 - Gradient not diminished
 - RNN
 - Gradient signal diminished for position $t' \ll t$
 - Truncated Back Propagation may kill the gradient flow from position t back to t' beyond truncation window

A key strength of the Transformer is that it enables learning long-range dependencies.

Number of parameters

In the Transformer, the Q, K, V matrices are first projected through $(d \times d)$ matrices, $\mathbf{W}_k, \mathbf{W}_Q, \mathbf{W}_V$

$$\begin{array}{ccccc} \text{out} & & \text{left} & & \text{right} \\ \hline Q & = & Q & * & \mathbf{W}_Q \\ \hline (T & & (T & & (d \times d \\ \times d) & & \times d) & &) \end{array}$$

$$|K| = |K| |\mathbf{W}_K| \quad |V| = |V| |\mathbf{W}_V| \quad (\bar{T} \times d) \quad |(\bar{T} \times d)| \quad (d \times d)$$

Each of the matrices, $\mathbf{W}_k, \mathbf{W}_Q, \mathbf{W}_V$, is $\mathcal{O}(d^2)$ parameters.

The Feed Forward Network is usually implemented by 2 **Dense** layers.

the first takes the length d attention output

- and creates d_{ffn} new features
- by custom: $d_{\text{ffn}} = 4 * d$

The second takes the length d_{ffn} output of the first and creates the final length d_{model} output.

Thus, each **Dense** layer has $\mathcal{O}(d^2)$ weights.

Number of operations

What about the number of operations ?

The Attention Lookup is computed via matrix multiplication

$$Q * K^T * V$$

$Q * K^T$ has $(T \times \bar{T})$ elements, each the result of d multiplications.

Thus: $\mathcal{O}(T^2 * d)$ multiplications.

The Self Attention layer attend to (transformed) inputs

- each element assumed size of d

The keys and values of the CSM implementing Attention are the size d input elements.

- Each attention lookup (dot product of query with a key) requires d multiplications.
- There are T key/value pairs in the CSM
- There are T attention units (one for each position, outputting $\mathbf{h}_{(t)}$)

Thus: $\mathcal{O}(T^2 * d)$ multiplications.

RNN calculations

Let's examine the RNN's number of operations and weights.

The RNN inputs $\mathbf{x}_{(t)}$ and outputs $\mathbf{h}_{(t)}$ of size d (same as Transformer).

- In the RNN $\mathbf{h}_{(t)}$ is also the latent state

Each step of the RNN updates the latent state $\mathbf{h}_{(t)}$ via the equation

$$\mathbf{h}_{(t)} = \phi(\mathbf{W}_{xh}\mathbf{x}_{(t)} + \mathbf{W}_{hh}\mathbf{h}_{(t-1)} + \mathbf{b}_h)$$

The weight matrices

$$\mathbf{W}_{xh} \text{ and } \mathbf{W}_{hh}$$

are of size

$$\mathcal{O}(d \times d)$$

- transforming length d vectors $(\mathbf{x}_{(t)}, \mathbf{h}_{(t-1)})$ into a length d vector $\mathbf{h}_{(t)}$

The multiplication of $(d \times d)$ weights matrices times a vector of length d

- requires d multiplications per element
- there are d elements in $\mathbf{h}_{(t)}$

Thus $\mathcal{O}(d^2)$ operations per time step.

There are T *sequential* time-steps

- $\mathcal{O}(T * d^2)$ total operations
- involving T sequential steps
 - steps are computed sequentially in the RNN, versus in parallel in the Transformer
- path length T as gradient flows backward through each of the T time steps

Complexity: summary

We also throw in a CNN for comparison

The detailed CNN math is given in a following section.

Type	Parameters	Operations	Sequential steps	Path length
CNN	$\mathcal{O}(k * d^2)$	$\mathcal{O}(T * k * d^2)$	$\mathcal{O}(T)$	$\mathcal{O}(T)$
RNN	$\mathcal{O}(d^2)$	$\mathcal{O}(T * d^2)$	$\mathcal{O}(T)$	$\mathcal{O}(T)$
Self-attention	$\mathcal{O}(d^2)$	$\mathcal{O}(T^2 * d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

Reference:

- [Transformer Scaling paper \(https://arxiv.org/pdf/2001.08361.pdf#page=6\)](https://arxiv.org/pdf/2001.08361.pdf#page=6)
- [Table 1 of Attention paper \(https://arxiv.org/pdf/1706.03762.pdf#page=6\)](https://arxiv.org/pdf/1706.03762.pdf#page=6)
- See [Stack overflow \(https://stackoverflow.com/questions/65703260/computational-complexity-of-self-attention-in-the-transformer-model\)](https://stackoverflow.com/questions/65703260/computational-complexity-of-self-attention-in-the-transformer-model) for correction of the number Operations calculated in paper

Transformer main point of comparison to the RNN

- fewer Sequential Steps: $\mathcal{O}(1)$ versus $\mathcal{O}(T)$
- operations: $\mathcal{O}(T^2 * d)$ versus $\mathcal{O}(T * d^2)$
 - more when sequences are long, i.e., $T > d$

But: because of the reduced number of sequential steps, Transformers

- can stack *many* (i.e., n_{layers}) blocks, each taking $\mathcal{O}(1)$ time
 - $\mathcal{O}(n_{\text{layers}})$ Sequential Steps total
- and still be less than the $\mathcal{O}(T)$ Sequential Steps of an RNN
- at the cost of increasing number of operations and parameters by $\mathcal{O}(n_{\text{layers}})$

Transformers consume larger number of parameters and operations through this factor of n_{layers} blocks.

CNN calculations

Here's the details of the math for the CNN

- path length T
 - each kernel multiplication connects only k elements of $\mathbf{x}$
 - since kernels overlap inputs, can't parallelize, hence $\mathcal{O}(T/k)$ path length
 - can reduce to $\log(T)$ with tree structure
- Parameters
 - kernel size k
 - number of input channels = number of output channels = d
 - $k * d$ parameters for kernel of one channel
 - $\mathcal{O}(k * d^2)$ parameters for kernel for all d output channels
- Operations
 - for a single output channel: k per input channel
 - There are d input channels, so $k * d$ for each dot product of one output channel
 - There are d output channels, so $k * d^2$ per time step
 - T time steps so $\mathcal{O}(T * k * d^2)$ number of operations

A free lunch ? Almost !

Transformers sound almost too good to be true

- Faster compute (through reduced number of Sequential steps)
- Constant Path Length
 - Better able to capture long range dependencies

Is there really such a thing as a free lunch ?

Almost.

In order to achieve the full benefit of reduced path length

- the operations across all T positions must be computed in parallel
- this involves a tremendous amount of simultaneous compute power
 - very expensive in hardware and power costs

In addition, *positional encoding* needs to be preserved at each layer

- to maintain relative ordering (e.g., for causal attention)
- more complicated than an RNN

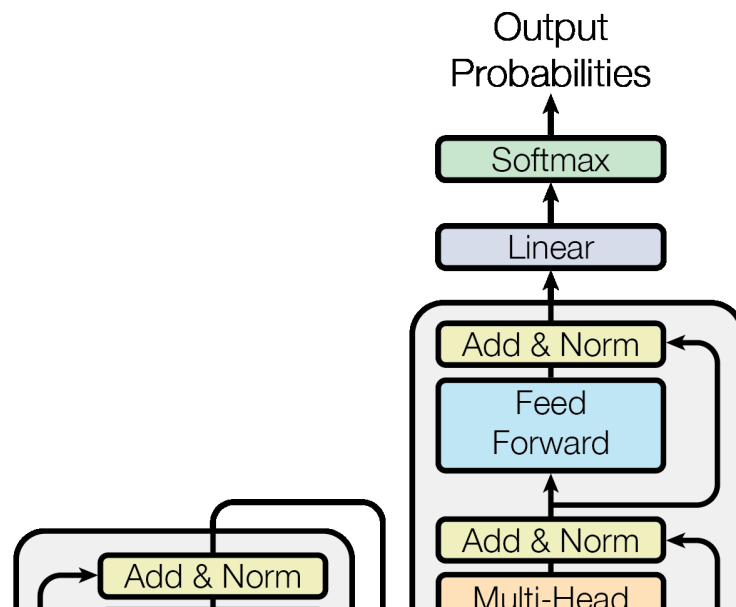
Detailed Encoder/Decoder Transformer architecture

There are other components of the Encoder and Decoder that we have yet to describe.

We will do so briefly.

The Transformer was introduced in the paper [Attention is all you Need](https://arxiv.org/pdf/1706.03762.pdf) (<https://arxiv.org/pdf/1706.03762.pdf>).

Transformer (Encoder/Decoder)



Embedding layers

We will motivate and describe Embeddings in the NLP module.

For now:

- an embedding is an encoding of a categorical value that is shorter than OHE

It is used in the Transformer to

- encode the input sequence of words
- encode the output sequence of words

Positional Encoding

The inputs are ordered (i.e., sequences) and thus describe a relative ordering relationship between elements.

But inputs to most layer types (e.g., Fully Connected) are unordered.

The Positional Encoding is a way of encoding the the relative ordering of elements.

To represent the relative position of each element in the sequence,

- we can pair the input element with an encoding of its position in the sequence.
 $\langle \mathbf{x}_{(t)}, \text{encode}(t) \rangle$

The box labeled "Positional Encoding" creates $\text{encode}(t)$.

The "+" concatenates the Input Embedding and Positional Encoding to create $\langle \mathbf{x}_{(t)}, \text{encode}(t) \rangle$.

If relative position is important, the NN can learn the values of $\text{encode}(t)$.

The encoding is subtle.

A fuller explanation is given in this [module \(Transformer_PositionalEmbedding.ipynb\)](#).

Self Attention layers (Encoder and Decoder)

The 3 arrows flowing into the Multi-Head Attention box

- **are identical**
- **are the inputs (after being Embedded and having Positional Encoding added)**

The Self-Attention layers for the Encoder and Decoder

- differ in that the Decoder uses Causal Masking versus no-masking for the Encoder
- Decoder can't "look ahead" at output $y_{t'}$ for $t' \geq t$
 - it hasn't been generated yet at test time step t
 - it is available at training time (via Teacher Forcing)
 - but shouldn't look at it during training time, in order for training to be similar to test time

Cross Attention layer (Decoder)

The two arrows flowing from the Encoder output are the keys and values of the CSM

The arrow flowing from the Self Attention layer is the query

- **The output of the Self Attention layer is the query used in Cross Attention**

Add and Norm

We have seen each of these layer types before

- Norm: Batch (or other) Normalization layers
- Add: the part of the residual network that joins outputs of multiple previous layers

The diagram shows an Encoder/Decoder pair.

You will notice that each element of the pair is different.

- It is possible to use each element independently as well.
- But first we need to understand the source of the differences and their implications.

What happens during training ?

Encoder

The Encoder uses self-attention

- So the keys and values of the CSM are derived directly from input sequence $\mathbf{x}_{(1..T)}$

During training, the Encoder

- learns a query, derived from input sequence $\mathbf{x}_{(1..T)}$
- learns weights for the Feed Forward Network

The Attention output

- is equal to a weighted combination of CSM values
 - i.e., weighted sum of input elements

The Feed Forward Network transforms the Attention output into Encoder output $\bar{\mathbf{h}}_{(t)}$.

Decoder

Similarly for the Decoder.

The Self-Attention layer CSM has keys and values that are incrementally constructed from the outputs $\hat{\mathbf{y}}_{(1..,t)}$ that have been created from the first t steps.

The Cross-Attention layer CSM has keys and values that are outputs $\bar{\mathbf{h}}_{(t)}$ of the Encoder.

During training, the Self-Attention layer learns to construct

- The query that is used for Self Attention.
 - attention to the inputs.
- The output of Self-Attention
 - the weighted sum of input positions
 - becomes the query that is used for Cross Attention.
- The query used for Cross Attention
 - attention to the Encoder outputs
- The output of Cross Attention
 - the weighted sum of Encoder outputs
 - becomes the input to the Feed Forward Network
- The weights of the Feed Forward Network
 - this is where "world knowledge" from training data is stored

Technical clarifications

Functional versus Sequential architecture

The architecture diagram is more complex than we have seen thus far.

In particular: data no longer strictly flows forward in a layer-wise arrangement !

- There are two independent sub-networks (Encoder and Decoder)
- Connection from the Encoder output to the middle of the Decoder (Cross-Attention)

Each of the Encoder and Decoder is an independent Functional model.

- not our familiar Sequential modles

The Encoder/Decoder pair combination is also constructed as a Functional model.

Since we have not yet addressed Functional Models, you may not be prepared to completely grasp the totality.

But hopefully you can absorb the concepts even without fully understanding the details.

Shared Transformer blocks across positions

The transformer blocks ("circles" in the diagram)

- are shared across all positions
- that is: the same computation (with shared parameters) is performed in parallel
- Thus, the number of parameters is not a function of sequence length T

Identifying $\hat{\mathbf{y}}_{(t)}$ with $\mathbf{h}_{(t)}$

The simplest RNN (corresponding to our diagrams) use the latent state $\mathbf{h}_{(t)}$ as the output $\hat{\mathbf{y}}_{(t)}$

$$\hat{\mathbf{y}}_{(t)} = \mathbf{h}_{(t)}$$

It is easy to add another NN to transform $\mathbf{h}_{(t)}$ into a $\hat{\mathbf{y}}_{(t)}$ that is different.

- We can add a NN to the Decoder RNN that implements a function D that transforms the latent state into an output.

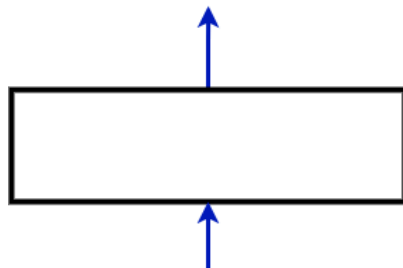
$$\hat{\mathbf{y}}_{(t)} = D(\mathbf{h}_{(t)})$$

Here is what the additional NN looks like:

Decoder output transformation: No attention

Decoder

$$\hat{\mathbf{y}}_{(1)}, \hat{\mathbf{y}}_{(2)}, \dots, \hat{\mathbf{y}}_{(T)}$$



In the context of the Transformer: we will assume the style of a single output $\mathbf{h}_{(t)}$

The reason for doing this:

- We can "stack" N Transformer layers (just as we can stack RNN layers)
- The output of the non-top layer j is $\mathbf{h}_{(t)}^{[j]}$, not the final $\mathbf{y}_{(t)}$
- We identify $\mathbf{y}_{(t)}$ as the output of the top layer $\mathbf{h}_{(t)}^{[N]}$
 - perhaps after a further processing

Furthermore:

Since the Encoder part is no longer a "loop"

- It is inaccurate to refer to the Encoder output $\bar{\mathbf{h}}_{(t)}$ as a "latent" state
- However, $\bar{\mathbf{h}}_{(t)}$ *is still* a summary of the input sequence
 - a summary of $\mathbf{x}_{(1..t)}$ when casual attention is used
 - a summary of $\mathbf{x}_{(1..\bar{T})}$ otherwise
- Out of bad habit we may continue to erroneously refer to $\bar{\mathbf{h}}$ and $\mathbf{h}$ as "latent" states

Conclusion

The Transformer architecture has come to dominate tasks with long sequences (e.g., NLP).

The operations of a Transformer occur in parallel for each position.

This allows us to leverage the compute time

- Use many stacked Transformer layers
- At time cost still less than a sequential RNN layer

Moreover, the constant path length means the gradients are less likely to vanish/explode for long sequences

- No need to truncate Back Propagation as in an RNN
- Long term dependencies between positions become feasible.

We pay for these advantages in terms of increasing

- number of operations
 - but they occur in parallel, so no increase in elapsed time
- number of weights

Thus, Transformer training is both compute and memory intensive.

- This limits the number of individuals/organizations able to train very large models.

In [2]: `print("Done")`

Done

